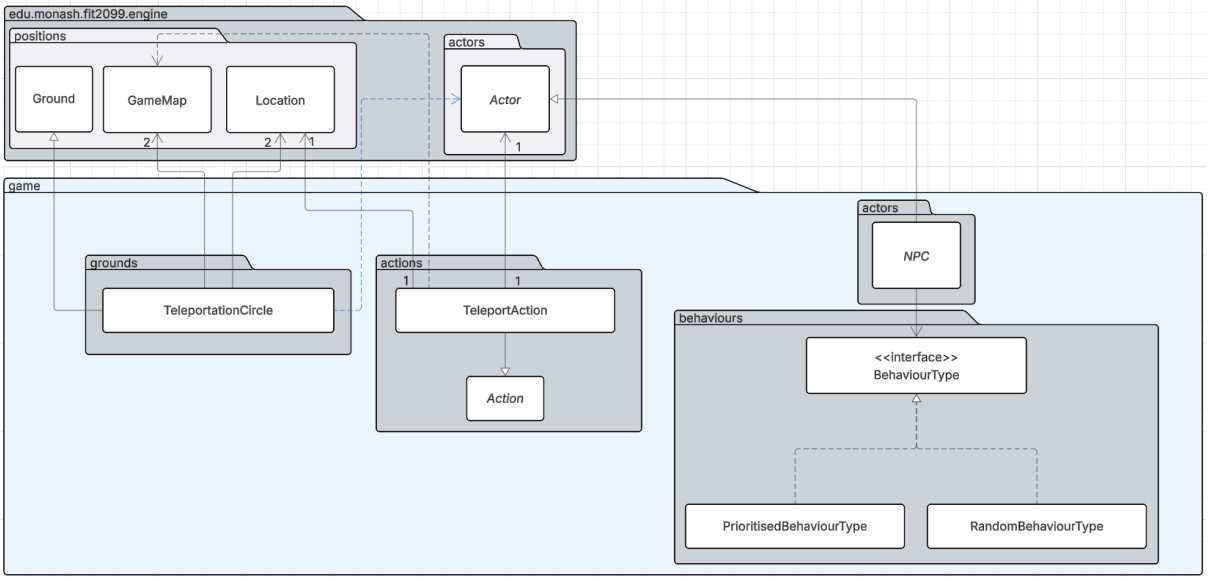
FIT2099 A3 Design Documentation

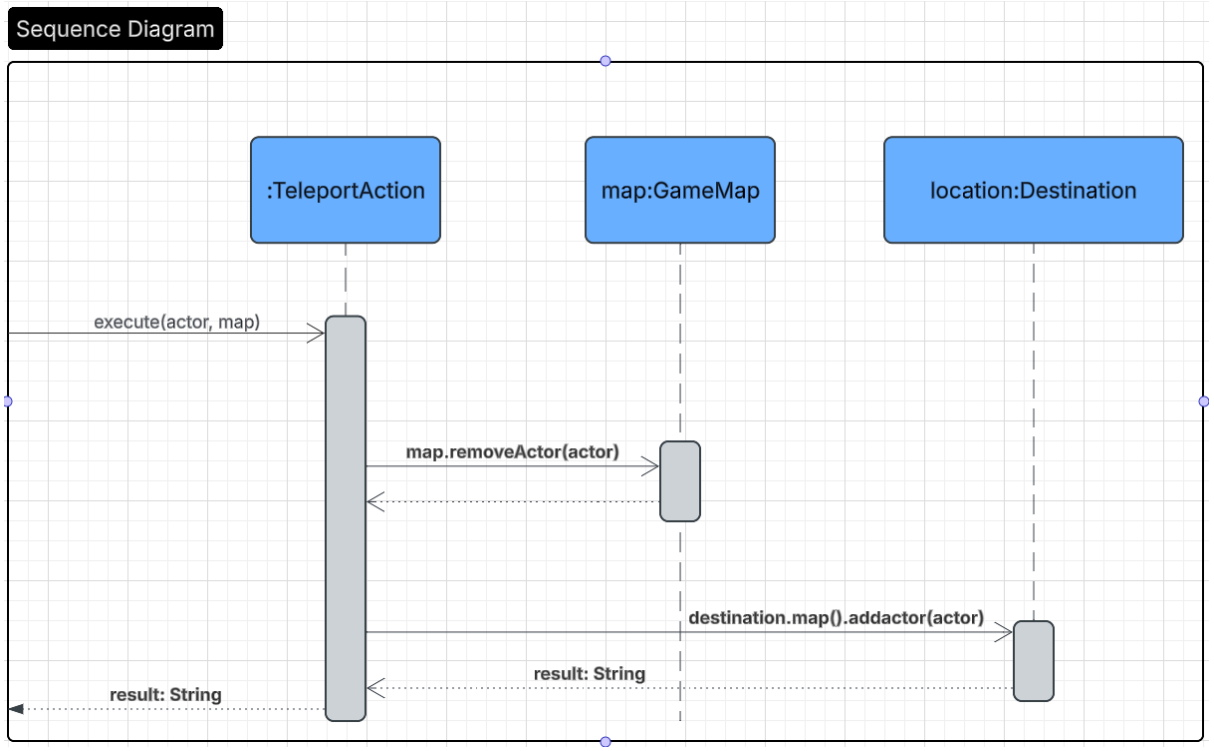
Contribution Log

REQ 1

UML:



Sequence:



Changes from A2

- NPC now contains a ‘BehaviourType’ attribute to determine whether the behaviour is prioritised or random.

TeleportationCircle

A specialized ground type that creates a network of connected teleportation points in the game world. Each circle knows its current location and destination, enabling bidirectional travel between points.

Responsibility:

- Manages teleportation point locations and destinations
- Offers teleport actions to actors standing on it
- Maintains the state of a single teleportation point

SOLID Principles:

- **Open-Closed Principle:** Extends Ground class cleanly, allowing for future specialized teleportation circles
- **Dependency Inversion Principle**: Depends on abstractions (Actor, GameMap, Location) rather than concrete implementations
- **Single Responsibility Principle:** Each method has a clear, focused purpose related to teleportation management

TeleportAction

Handles the teleportation logic when a player uses a teleportation circle. Manages the transition of actors between locations, including cross-map teleportation.

Responsibility:

- Executes the teleportation of an actor from one location to another
- Handles the removal and addition of actors between maps
- Provides appropriate action descriptions for the game interface

SOLID Principles:

- **Single Responsibility Principle:** Focused solely on teleportation execution logic
- **Liskov Substitution Principle:** Properly implements Action abstract class methods
- **Open-Closed Principle:** Can be extended for specialized teleportation behaviors

BehaviourType

Defines a strategy pattern interface for different behavior selection mechanisms, allowing for flexible NPC decision-making systems.

Responsibility:

- Defines the contract for behavior selection algorithms

- Enables runtime switching between different behavior types

SOLID Principles:

- **Interface Segregation Principle:** Minimal interface with just selectAction
- **Dependency Inversion Principle:** High-level modules depend on this abstraction
- **Open-Closed Principle:** New behavior types can be added without modifying existing code

PrioritisedBehaviourType

Implements ordered behaviour selection where NPCs attempt behaviours in strict priority order, ensuring critical actions are tried before fallback behaviors.

Responsibility:

- Orders behaviours by priority
- Attempts behaviours sequentially until finding a valid action

SOLID Principles:

- **Liskov Substitution Principle:** Properly implements BehaviourType interface
- **Single Responsibility Principle:** Focused on priority-based selection
- **Dependency Inversion Principle:** Works with abstractions rather than concrete implementations
- **Open-Closed Principle:** Can be extended for different priority implementations

RandomBehaviourType

Implements behaviour selection where NPCs randomly choose one behaviour.

Responsibility:

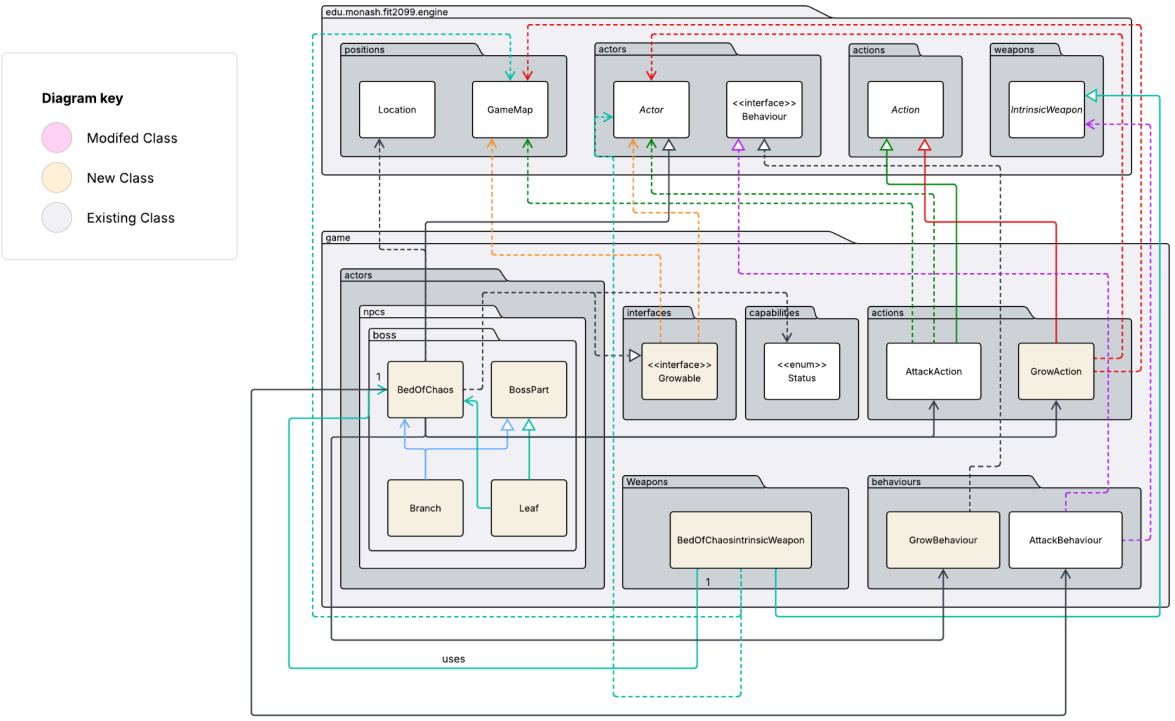
- Randomly selects a single behaviour to attempt

SOLID Principles:

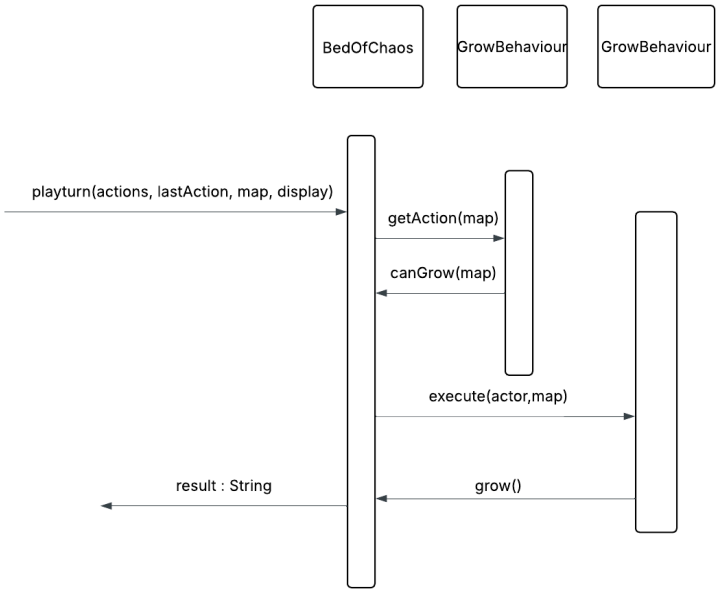
- **Single Responsibility Principle:** Focused purely on random selection
- **Liskov Substitution Principle**: Properly implements BehaviourType interface
- **Dependency Inversion Principle:** Works with abstractions
- **Interface Segregation Principle:** Implements only the necessary method from the interface

REQ 2

UML DIAGRAM:



Sequence Diagram:



This diagram captures the interactions between BedOfChaos, GrowBehaviour, and GrowAction, showing how growth is triggered and executed in a step-by-step manner, aligning with the game's SOLID-based design principles.

BedOfChaos Class
The BedOfChaos class represents the core of the boss encounter, managing its attack logic, growth, and dynamic damage system. It extends the abstract Actor class and implements the Growable interface to control its growth behaviour.

Responsibilities:

- Maintains a list of attached parts (Branch and Leaf) that contribute to its total attack damage.
- Decides whether to grow (via the grow() method) or attack based on proximity to the player.
- Provides a clear string representation of its current state (HP, parts, total damage).

SOLID Alignment:

- Single Responsibility (SRP): Handles only boss-specific combat and growth behaviour.
- Open/Closed (OCP): Can accommodate new parts (e.g., advanced branches or leaves) without modifying core logic.
- Liskov Substitution (LSP): Can be extended with new types of bosses using the same interface (Growable) without breaking functionality.

BossPart Abstract Class
The BossPart abstract class defines a base for all parts that contribute to the boss's attack damage.
Responsibilities:

- Holds the basic damage contribution for the part.
- Provides a method to retrieve damage contribution (getAttackDamage()).

SOLID Alignment:

- Single Responsibility (SRP): Handles only the damage contribution of a boss part.
- Open/Closed (OCP): Allows adding new types of parts with different damage logic by subclassing.

Branch Class
The Branch class is a concrete subclass of BossPart representing the boss's growing parts.
Responsibilities:

- Adds recursive growth behaviour: branches can grow other branches or leaves until a leaf is grown (ending the turn).
- Accumulates damage from all sub-parts recursively.
- Heals the boss whenever a leaf is grown.

SOLID Alignment:

- Single Responsibility (SRP): Manages branch growth and sub-part damage calculation.
- Open/Closed (OCP): Allows adding new sub-parts or growth rules by extending its logic.
- Liskov Substitution (LSP): Fully interchangeable wherever a BossPart is expected.

Leaf Class

The Leaf class is a simple subclass of BossPart that represents a damage contribution point.
Responsibilities:

- Adds a fixed damage contribution to the boss.
- Heals the boss by a flat amount.

SOLID Alignment:

- Single Responsibility (SRP): Only provides a small damage boost to the boss.

BedOfChaosIntrinsicWeapon Class
The BedOfChaosIntrinsicWeapon represents the boss's attack, combining its base damage and all parts' accumulated damage into a single attack.
Responsibilities:

- Computes total damage at each attack based on the current state of the boss.
- Applies hit/miss logic based on a set hit rate.

SOLID Alignment:

- Single Responsibility (SRP): Handles only attack logic for the boss.
- Open/Closed (OCP): Can be extended for alternative attack effects (e.g., different hit rates or damage calculations).

GrowBehaviour and GrowAction Classes
GrowBehaviour implements the Behaviour interface, deciding whether the boss should grow on its turn if no players are nearby.
GrowAction executes the actual growth logic.
Responsibilities:

- GrowBehaviour: Checks whether growth is allowed based on proximity to the player.
- GrowAction: Executes the boss's growth logic.

SOLID Alignment:

- SRP: Each class handles a single piece of logic: GrowBehaviour decides, GrowAction executes.
- OCP: Can extend growth actions or behaviours without modifying existing logic.

Status Enum and Status Effect
The Status enum allows the game to mark actors as hostile or non-hostile, controlling which

actors the boss attacks.

Responsibilities:

- Manages combat interactions (hostility detection).

SOLID Alignment:

- Open/Closed (OCP): Can add new statuses (e.g., VULNERABLE, DEFENSIVE) easily.

Evaluation of Current Design Pattern

Pros:

- Extensibility: Easy to add new parts (e.g., SpikedBranch) or modify boss behaviour with minimal code changes.
- Modularity: Each behaviour (attack, growth) is encapsulated in its own class.
- Encapsulation: Each part manages its own damage and growth.
- Decoupling: Damage and growth are separated from the boss's core logic.

Cons / Limitations:

- Recursive Growth: Branch growth could become deep, potentially leading to performance concerns if not managed.
- Distributed Responsibility: Growth logic is split between BedOfChaos and Branch classes, requiring coordination.

Design Alternatives

Centralised Growth Manager

An alternative would be to have a dedicated GrowthManager class that oversees the growth of all parts.

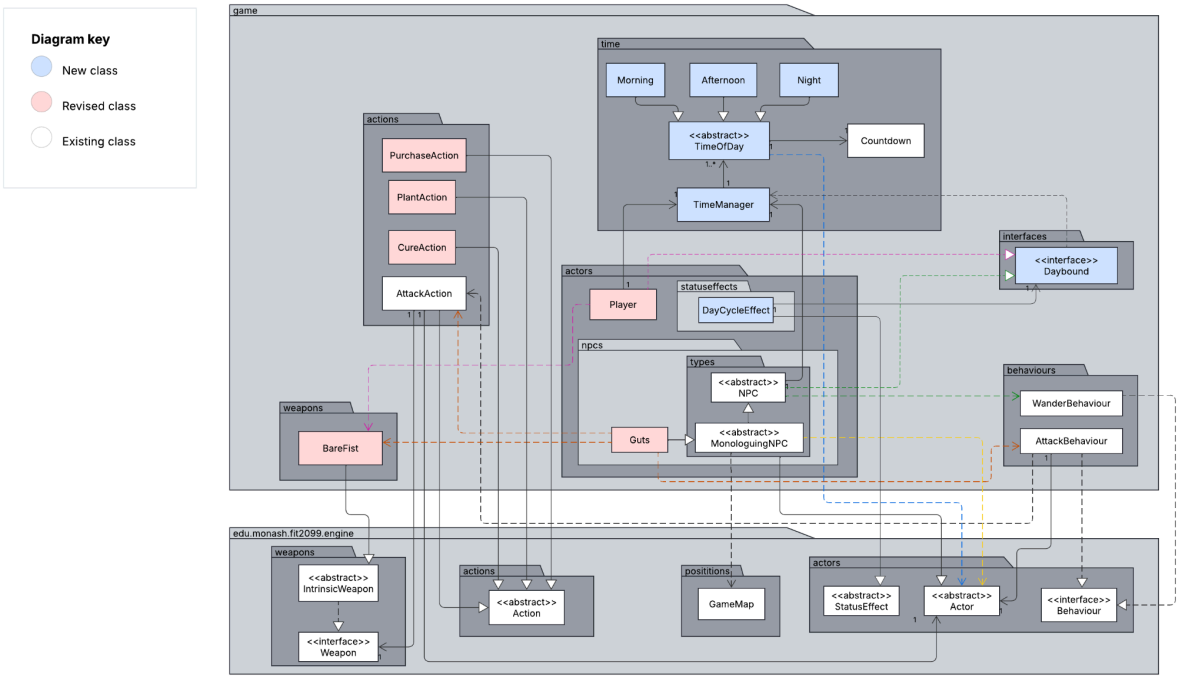
Pros:

- Centralises growth logic and state.
- Could coordinate more complex growth patterns.

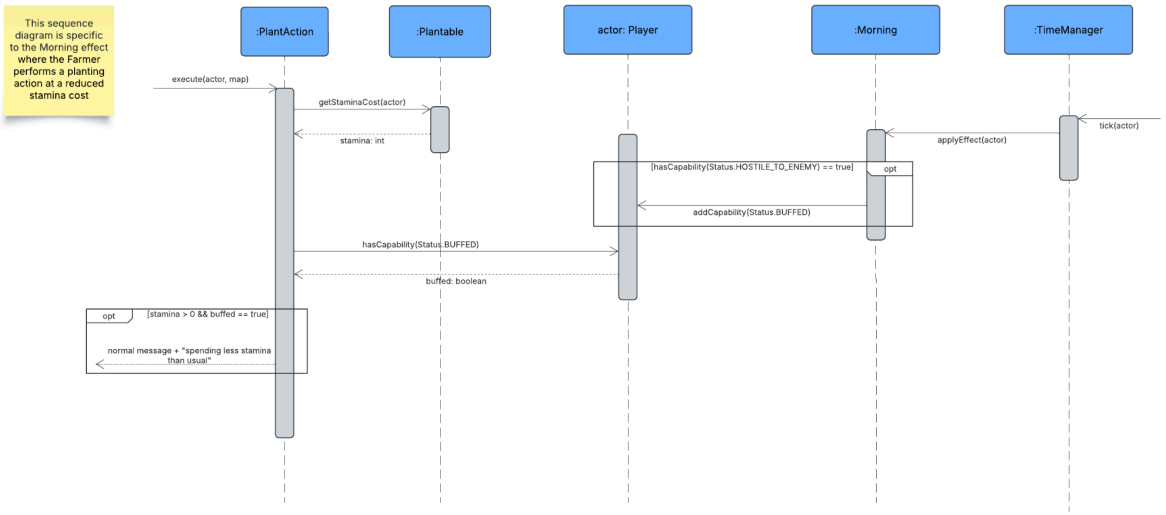
Cons:

- Introduces coupling between the GrowthManager and the boss parts.
- Reduces encapsulation of growth logic in parts themselves.

REQ 3



The UML diagram represents the day/night feature extension to the Elden Thing game architecture. The system was designed to dynamically alter the world state and Actor behaviour based on the time of day.



This sequence diagram demonstrates the specific effect of the Morning TimeOfDay, where the Farmer performs a planting action at a reduced stamina cost due to the active Morning buff. It highlights how the Status.BUFFED capability is checked during the PlantAction, resulting in a stamina cost reduction of 25%.

TimeManager Class

The TimeManager is a high-level class responsible for orchestrating the progression of time. It holds a List<TimeOfDay> representing the day cycle and tracks the current index in the cycle to transition across phases based on configurable round durations.

SOLID Alignment:

- **Single Responsibility (SRP)**: Handles only the management of time within a day.
- **Open/Closed (OCP):** Can be extended with new TimeOfDay types without modifying existing code.
- **Dependency Inversion (DIP)**: Depends on abstract TimeOfDay, not its concrete implementations.

TimeOfDay Abstract Class

The TimeOfDay abstract class encapsulates the behaviour of a single period (e.g., Morning, Afternoon, Night) and is associated with a Countdown to track how many rounds the period lasts.

- **Abstract Method**: applyEffect(Actor actor) allows subclasses to define period-specific effects.
- **Lifecycle Methods**: tick(), hasPassed(), and reset() manage temporal transitions.

SOLID Alignment:

- **Liskov Substitution (LSP)**: Any subclass (e.g., Morning, Night) can be used wherever TimeOfDay is expected.
- **Open/Closed (OCP)**: Allows new periods (e.g., Dusk, Eclipse) without modifying base logic.

Morning, Afternoon, Night – Concrete TimeOfDay Classes

Each TimeOfDay subclass defines unique world behaviour by overriding applyEffect.

Morning

- **Effect:** Stamina costs for actions are reduced by 25% for the Farmer.
- **Implementation:**
    - Farmer receives Status.BUFFED.
    - Actions like planting and curing implement getStaminaCost(Actor) via Plantable and Curable interfaces.
    - If Status.BUFFED is present, stamina cost is scaled (e.g., original * 0.75f).

Afternoon

- **Effect:** Only during this time can the Farmer trade with Merchants.
- **Implementation:**
    - Farmer is granted Ability.PURCHASE.
    - PurchaseAction checks for this ability before executing.
    - Removed in other periods to enforce restriction.

Night

- **Effect:** Aggression increases for the Farmer and Guts.
- **Implementation:**
    - Actors are given Ability.ATTACK and Status.AGGRESSIVE.
    - Guts' allowableActions() includes AttackAction only when opponents are aggressive.
    - Damage multiplier is increased conditionally:
        - **+30%** when on or near Blight
        - **+15%** otherwise

DayCycleEffect Class

The DayCycleEffect class extends the StatusEffect abstract class, enabling the passive transition of phases within a day. It is introduced as a StatusEffect in Player and NPC classes and calls TimeManager.tick() before an actor's turn to exert the effect of that period on the actor and progress the day.

SOLID Alignment:

- **DIP:** The DayCycleEffect depends on the Daybound interface, not concrete implementations of actors, reducing coupling.

Daybound Interface

Marks actors/entities as being linked to a TimeManager, ensuring the DayCycleEffect can find and operate on the correct cycle controller.

SOLID Alignment:

- **ISP:** Daybound clearly defines what it means for an actor to be time-aware without forcing unrelated classes to implement unnecessary logic.
- **OCP:** Can introduce new methods specific to Daybound entities easily by defining the method in the interface

Status and Ability Enums

To facilitate clean and extensible toggling of behaviour:

- **Status**: Describes temporal states like BUFFED, AGGRESSIVE.
- **Ability**: Defines what an actor can do (e.g., ATTACK, PURCHASE).

These enums provide a decoupled, capability-based design. Instead of hardcoding behaviour into classes, the system checks and responds to toggled capabilities, consistent with the OCP.

Evaluation of Current Design Pattern

Pros

- **Extensibility**: New periods and effects can be added with minimal changes to existing code.
- **Modularity**: Each time period is self-contained.
- **Reusability**: Status and Ability enums are general and reusable in other systems.
- **Encapsulation**: Each behaviour is encapsulated in its respective class.

Cons / Limitations

- **Actor Awareness**: Actors need awareness of status/effects set by time (e.g., stamina check logic must be in the action itself).
- **Indirect Control Flow**: Effects are not centralised; they require distributed implementation across affected entities (e.g., in Plantable, Curable, etc.).

Design Alternatives

Centralised Tick in GameMap

Instead of ticking time from DayCycleEffect, the game loop (GameMap.tick()) could directly manage time progression.

Pros:

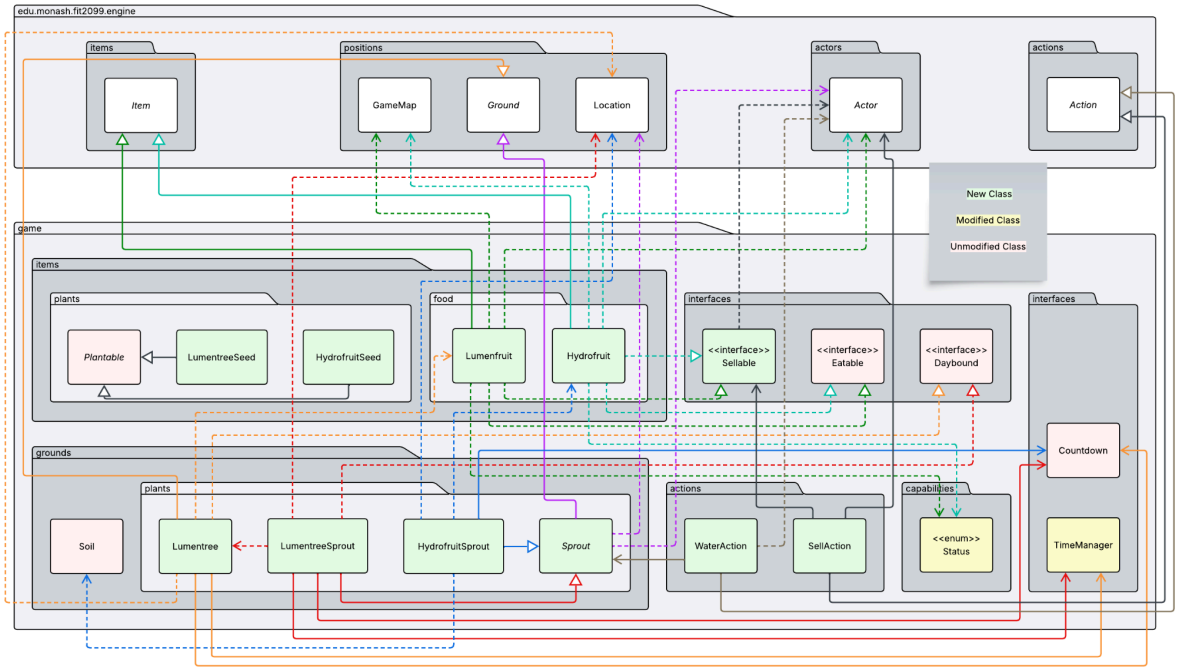
- Simpler to reason about
- Avoids coupling to actors

Cons:

- Violates Single Responsibility (mixes actor logic and time management)
- Harder to apply time-specific effects directly to actors.

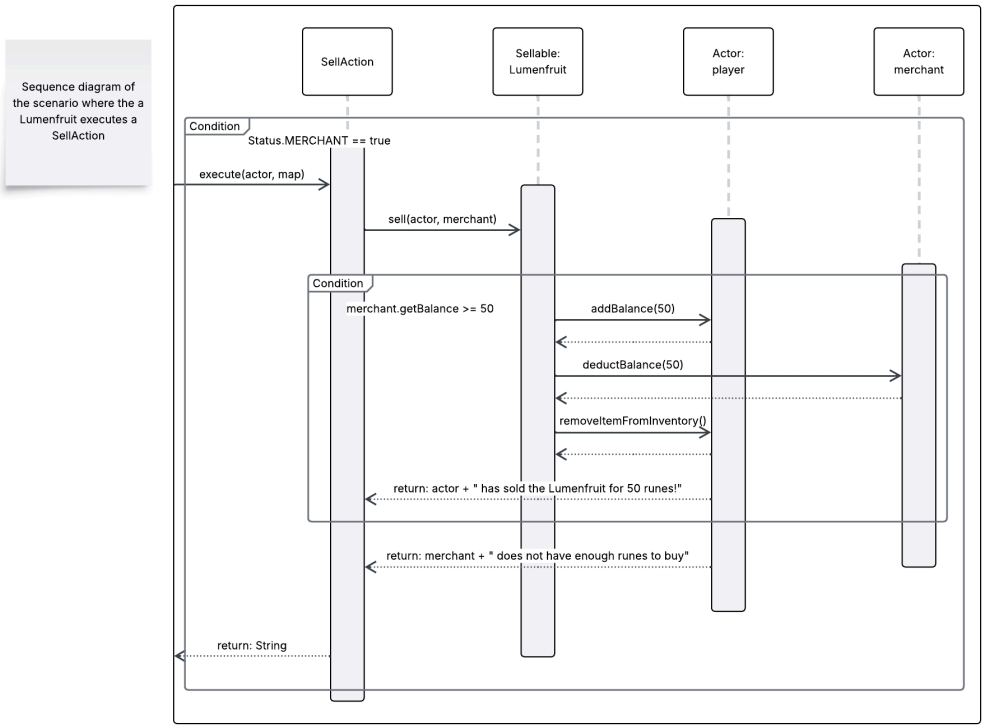
REQ 4

UML Class Diagram:



This UML diagram represents an extension to Elden Thing where two new seeds LumentreeSeed and HydrofruitSeed can be planted and grown into crops. Once planted, they become Sprouts which only become fully grown after condition is met. They both also produce fruits once grown, and the produce can be sold to merchants.

SellAction Sequence Diagram:



WaterAction Class

The `Sprout` abstract class is an attribute of the `WaterAction` class, and is what the methods and behaviours of the `WaterAction` class are carried out on.

- By using a `Sprout` abstract class, `WaterAction` is able to work without implementing individual subclasses, which adheres to the **Dependency Inversion Principle (DIP)**
- `WaterAction` has a `Sprout` attribute which allows all sprouts to have `WaterAction` as part of their action list, following the **Open/Closed Principle (OCP)**
- The `WaterAction` class is solely responsible for the action of watering a `Sprout`, which follows the **Single Responsibility Principle (SRP)**

SellAction Class

The `SellAction` class contains an `Sellable` interface and a `merchant` Actor as attributes, which the methods and behaviours are acted on.

- As the `SellAction` class is responsible for the action of an `Sellable` being sold to a `merchant`, it follows the **Single Responsibility Principle (SRP)**
- `SellAction` utilises an `Sellable` interface instead of using individual classes, which follows the **Dependency Inversion Principle (DIP)**
- `SellAction` can handle and build upon any `Sellable` without extra modification, which adheres to the **Open/Closed Principle (OCP)**

Changes Made to the MerchantOffer and PurchaseAction Classes

Originally, a `Merchant` interface was used to differentiate between a merchant `Actor` and other actors. However, this resulted in the `completePurchase()` method in `MerchantOffer` to interact one-sidedly with the player, as the merchant was not considered an `Actor`.

Instead, I modified the code so that a `MERCHANT` status was used for differentiation instead, and replaced the `Merchant` attributes in `MerchantOffer` and `PurchaseAction` with `Actor`.

Pros of Removing the Merchant Interface:

- Allows the merchants to have their balance added to or deducted from, creating a better market system. This allows those with the status `MERCHANT` to be passed as

an `Actor` in the future, making them substitutable which follows the **Liskov Substitution Principle (LSP)**.
- Previously, `MerchantOffer` and `PurchaseAction` used a `Merchant` interface, which mixes actor and `Merchant` logic. By separating the status and behaviour with `Status.MERCHANT`, we follow the **Single Responsibility Principle (SRP)**.
- The only method in the `Merchant` interface was a `getOffers()` method which was never used. By removing the interface, the design is simplified which resolves the **Interface Segregation Principle (ISP)** violation.

Cons of Removing the Merchant Interface:

- Using a `Merchant` interface allows the merchant actors to be immediately identifiable, while the `MERCHANT` status is less obvious, causing an **Open/Closed Principle (OCP)** tradeoff due to less clarity.

Sprout Abstract Class

The `Sprout` class is an abstract class extending the `Ground` abstract class, and contains the `WaterAction` as part of its action list. In addition, the `Sprout` is also able to be watered and grown through the `water()` and `grow()` methods.

`Sprout` was originally implemented as an interface which only provided the base methods `water()` and `grow()`. However, it was later changed to an abstract class as it was determined to follow SOLID principles better, and `HydrofruitSprout` and `LumentreeSprout` were implemented as its subclasses.

Pros of Changing Sprout to an Abstract Class:

- Avoids repeating code in future implementations of other `Sprout` classes as it has its own `allowableActions` list, which adds `WaterAction` to the action list - **Don't Repeat Yourself (DRY)**
- By making `Sprout` an abstract class, it is easier to extend with subclasses, which adheres to the **Open/Closed Principle (OCP)**
- By depending on an abstract `Sprout` class instead of an interface, `Sprout` is able to hold attributes such as `Countdown` which allows the logic to be managed in one place, following the **Single Responsibility Principle (SRP)**

Cons of Changing Egg to an Abstract Class:

- As Java only allows a subclass to extend one superclass, this means that all future sprouts must inherit from the `Sprout` class, while interfaces allow multiple inheritance. This can cause limitations which risks the **Interface Segregation Principle (ISP)**

HydrofruitSprout Class

The `HydrofruitSprout` is an extension of `Sprout`, which means it doesn't need to add `WaterAction` to its action list as it is inherited. It grows into a `Hydrofruit` after 7 turns. Additionally, watering it 3 or more times allows it to produce double the fruit.

- It utilises a `Countdown` attribute to keep track of the turns until growing, which reuses methods rather than repeating code - **Don't Repeat Yourself (DRY)**
- By overriding the `water()` method and calling `super.water()`, it reuses the code as well as extends behavior, adhering to **Don't Repeat Yourself (DRY)** and **Liskov Substitution Principle (LSP)**.

LumentreeSprout and Lumentree Classes

Both `LumentreeSprout` and `LumentreeSprout` implement the `Daybound` interface and use a `TimeManager` to track day-night cycles. The growth only occurs during the day, while the act of producing a fruit only occurs at night.

- The use of `TimeManager` and a countdown encapsulates state and ticking behavior, adhering to the **Single Responsibility Principle (SRP)**.
- The fruit production is driven by a `Countdown` that resets upon successful production, reusing countdown logic and avoiding duplication — **Don't Repeat Yourself (DRY)**.
- The clear separation of responsibilities (growth in `LumentreeSprout`, production in `Lumentree`) supports maintainability and **Single Responsibility Principle (SRP)**.

Hydrofruit and Lumenfruit Classes

Both `Hydrofruit` and `Lumenfruit` extend `Item` and implement the `Eatable` and `Sellable` interfaces, allowing them to be consumed or sold by actors.

- Their `allowableActions` method dynamically builds context-based actions (`EatAction`, `SellAction`), which demonstrates flexible and extensible behavior — **Open/Closed Principle (OCP)**.
- The separation of `eatenBy()` and `sell()` methods respects **Single Responsibility Principle (SRP)** by clearly defining what happens in each interaction.
- The use of interfaces (`Eatable`, `Sellable`) ensures that they are decoupled from the consumers of those actions, such as `EatAction` and `SellAction`, following the **Dependency Inversion Principle (DIP)**.

- `Hydrofruit` produces a seed upon consumption, embedding unique logic while reusing interface contracts — this supports **Liskov Substitution Principle (LSP)** and keeps the codebase extensible.